

Министерство цифрового развития

Федеральное государственное общеобразовательное учреждение высшего
образования

«Сибирский Государственный Университет Телекоммуникаций и
Информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Вариант 2.

Отчет по практической работе №2.

«Модульное тестирование программ на языке Python средствами Visual Studio
Code»

Работу выполнил студент группы ИП213:

Жуйков С. А.

Соколовский Н. А.

Терновский Д. Р.

Работу проверил ст. преподаватель кафедры ПМиК:

Агалаков А. А.

г. Новосибирск, 2025 г

Цель работы

Цель: сформировать практические навыки разработки модульных тестов для библиотек классов Python и выполнения модульного тестирования с помощью средств автоматизации unittests.

Задание

Разработайте на языке Python класс, содержащий функции в соответствии с вариантом задания. Разработайте тестовые наборы данных для тестирования функций класса, по критерию C1. Протестируйте созданный класс с помощью средств автоматизации модульного тестирования unittests. Проанализируйте результаты выполненных тестов по объёму покрытия тестируемого кода. Напишите отчёт о результатах проделанной работы.

Вариант задания – 2

Поиск максимума из двух чисел. Функция получает двумерный массив вещественных переменных A. Отыскивает и возвращает сумму значений компонентов массива, у которых сумма значений индексов равна максимальному значению второго индекса (индекса столбца).

Пояснение для массива размером 2 на 2: Сумма = $A[0,2] + A[2,0] + A[1,1]$.

Функция получает двумерный массив вещественных переменных A.

Отыскивает и возвращает минимальное значение компонентов массива, лежащих на и выше побочной диагонали.

Листинг программы

matrix_functions.py

```
class MatrixFunctions:

    @staticmethod
    def find_max_of_two(a, b):
        return a if a > b else b

    @staticmethod
    def sum_elements_with_index_sum_equal_max_column(A):
        if not A or not A[0]:
            return 0

        rows = len(A)
        cols = len(A[0])
        max_col_index = cols - 1

        total_sum = 0.0
        for i in range(rows):
            for j in range(cols):
                if i + j == max_col_index:
                    total_sum += A[i][j]

        return total_sum

    @staticmethod
    def min_above_secondary_diagonal(A):
        if not A or not A[0]:
            return None

        n = len(A)
        min_value = float('inf')

        for i in range(n):
            for j in range(n):
                if i + j <= n - 1:
                    if A[i][j] < min_value:
                        min_value = A[i][j]

        return min_value
```

test_matrix__functions.py:

```
import unittest
from matrix_functions import MatrixFunctions

class TestMatrixFunctions(unittest.TestCase):

    def setUp(self):
        self.mf = MatrixFunctions()
```

```

def test_find_max_of_two_positive(self):
    self.assertEqual(self.mf.find_max_of_two(5, 3), 5)
    self.assertEqual(self.mf.find_max_of_two(2, 8), 8)

def test_find_max_of_two_negative(self):
    self.assertEqual(self.mf.find_max_of_two(-1, -5), -1)
    self.assertEqual(self.mf.find_max_of_two(-10, -3), -3)

def test_find_max_of_two_equal(self):
    self.assertEqual(self.mf.find_max_of_two(7, 7), 7)
    self.assertEqual(self.mf.find_max_of_two(0, 0), 0)

def test_sum_elements_2x2(self):
    matrix = [
        [1.0, 2.0],
        [3.0, 4.0]
    ]

    self.assertAlmostEqual(self-
.mf.sum_elements_with_index_sum_equal_max_column(matrix), 5.0)

def test_sum_elements_3x3(self):
    matrix = [
        [1.0, 2.0, 3.0],
        [4.0, 5.0, 6.0],
        [7.0, 8.0, 9.0]
    ]

    self.assertAlmostEqual(self-
.mf.sum_elements_with_index_sum_equal_max_column(matrix), 15.0)

def test_sum_elements_empty_matrix(self):
    self.assertEqual(self-
.mf.sum_elements_with_index_sum_equal_max_column([]), 0)
    self.assertEqual(self-
.mf.sum_elements_with_index_sum_equal_max_column([[]]), 0)

def test_min_above_secondary_diagonal_2x2(self):
    matrix = [
        [5.0, 2.0],
        [1.0, 8.0]
    ]

    self.assertAlmostEqual(self.mf.min_above_secondary_diagonal(ma-
trix), 1.0)

def test_min_above_secondary_diagonal_3x3(self):
    matrix = [
        [10.0, 5.0, 3.0],
        [2.0, 8.0, 6.0],
        [1.0, 4.0, 9.0]
    ]

```

```

        self.assertEqual(self.mf.min_above_secondary_diagonal(matrix), 1.0)

    def test_min_above_secondary_diagonal_empty(self):
        self.assertIsNone(self.mf.min_above_secondary_diagonal([]))
        self.assertIsNone(self.mf.min_above_secondary_diagonal([[]]))

if __name__ == '__main__':
    unittest.main()

```

Результаты работы модульных тестов

```

(cov) fluke@DESKTOP-ESF39FT:~/MTP/mtp2/MTP/stp2$ coverage report
Name                               Stmts  Miss  Cover
-----
matrix_functions.py                 29      0   100%
test_matrix_functions.py            34      1    97%
-----
TOTAL                               63      1    98%

```

Рис. 1 — Результат выполнения модульных тестов.

Результаты покрытия разработанного кода тестами

<u>Files Functions Classes</u>					
coverage.py v7.10.7, created at 2025-10-16 11:45 +0700					
File	class	statements	missing	excluded	coverage
matrix_functions.py	MatrixFunctions	22	0	0	100%
matrix_functions.py	(no class)	7	0	0	100%
test_matrix_functions.py	TestMatrixFunctions	19	0	0	100%
test_matrix_functions.py	(no class)	15	1	0	93%
Total		63	1	0	98%

Рис. 2 — Результат покрытия классов модульных тестов.

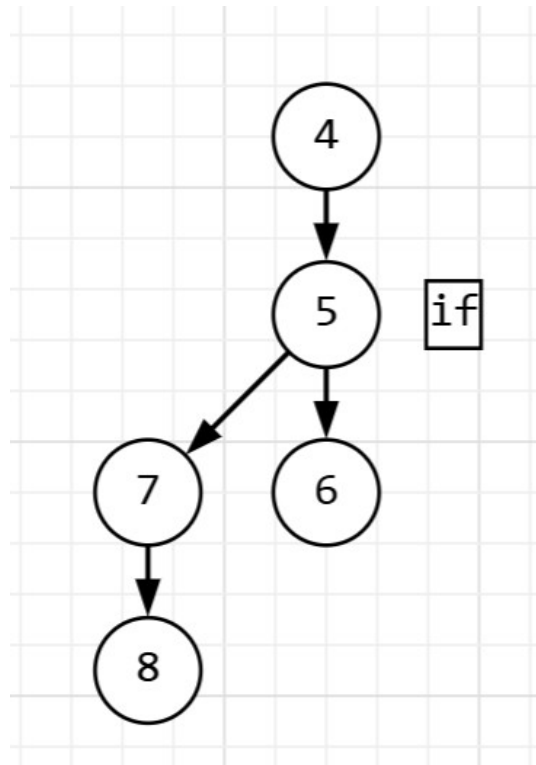
Files Functions Classes				
coverage.py v7.10.7, created at 2025-10-16 11:45 +0700				
File	function	statements	missing	excluded coverage
matrix_functions.py	MatrixFunctions.find_max_of_two	1	0	0 100%
matrix_functions.py	MatrixFunctions.sum_elements	11	0	0 100%
matrix_functions.py	MatrixFunctions.min_secondary	10	0	0 100%
matrix_functions.py	(no function)	7	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.setUp	1	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.test_find_max_of_two_positive	2	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.test_find_max_of_two_negative	2	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.test_find_max_of_two_equal	2	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.test_sum_elements_2x2	2	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.test_sum_elements_3x3	2	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.test_sum_elements_empty_matrix	2	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.test_min_secondary_2x2	2	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.test_min_secondary_3x3	2	0	0 100%
test_matrix_functions.py	TestMatrixFunctions.test_min_secondary_empty	2	0	0 100%
test_matrix_functions.py	(no function)	15	1	0 93%
Total		63	1	0 98%

Рис. 3 — Результат покрытия функций модульных тестов.

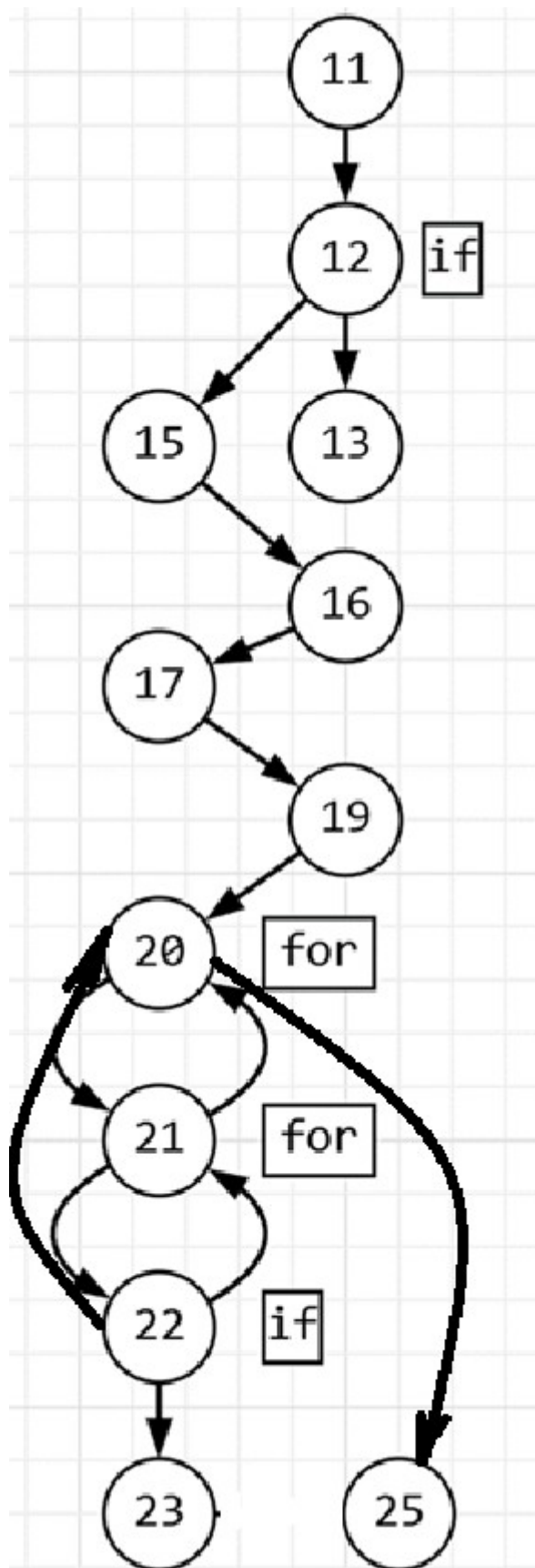
Files Functions Classes				
coverage.py v7.10.7, created at 2025-10-16 11:45 +0700				
File	statements	missing	excluded	coverage
matrix_functions.py	29	0	0	100%
test_matrix_functions.py	34	1	0	97%
Total	63	1	0	98%

Рис. 4 — Результат покрытия файлов модульных тестов.

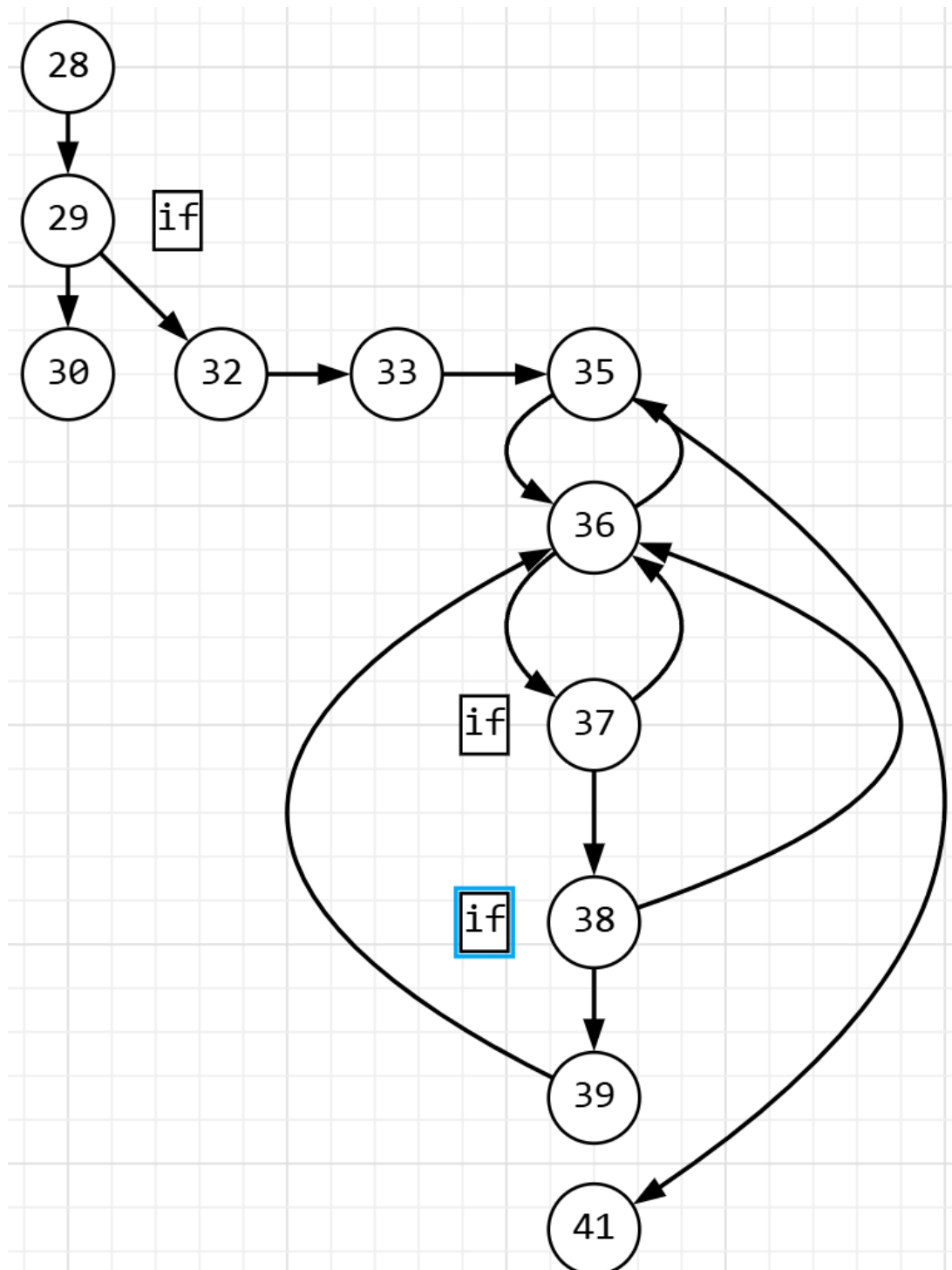
УГП



Управляющий граф для метода `find_max_of_two`



Управляющий граф для метода sum_elements



Управляющий граф для метода min_secondary